

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development | **Course Code:** 23CM4121

Observation Task No.: 6 | **Question No.:** 2 | **Course Outcome:** CO-2

Student Name: J Satya Srikar | **Roll No.:** A24126552259 | **Class:** CSM-B

Date: 06/10/2026 | **Lab Session:** Full Stack Web Development Laboratory

1. TITLE

Node.js Interactive File Management: File System (fs) Operations with User Input

2. AIM

To develop a Node.js console application that accepts a filename and text content from the user and performs sequential file operations using the built-in fs module: creating/writing the file, reading its contents, appending additional user content, and displaying the final merged file contents.

3. OBJECTIVES & SYSTEM REQUIREMENTS

Key Learning Objectives (CO-2):

- Master the Node.js fs (File System) module for basic server-side disk I/O.
- Accept interactive user inputs via the Node.js readline standard library module.
- Execute synchronous file creation using fs.writeFileSync() and verify byte allocation.
- Retrieve and decode file contents into UTF-8 text strings via fs.readFileSync().
- Append continuous text streams via fs.appendFileSync() without overwriting prior data.
- Analyze error handling boundaries (e.g., ENOENT directory errors) in server scripts.

System & Environment Requirements:

- **Hardware:** 64-bit PC / Laptop, 4 GB RAM, 10 GB disk space.
 - **Software:** Node.js runtime (v18+ or v20+), PowerShell / Command Prompt, VS Code.
-

4. THEORY & CONCEPTS (PART 1)

Node.js runtime environment executes JavaScript outside browser boundaries, granting access to operating system facilities such as secondary storage, network sockets, and process I/O. The built-in fs module provides an API for interacting with the file system based on standard POSIX functions.

When writing CLI utilities or administrative tools, applications frequently accept interactive user parameters. The built-in readline module facilitates reading streams line-by-line from process.stdin. By pairing readline with synchronous fs operations, developers can create reliable, sequential file management utilities with deterministic execution order.

4. THEORY & CONCEPTS (PART 2: FILE I/O LIFECYCLE & BUFFER DECODING)

The four fundamental file operations implemented include:

- **1. Create & Write (fs.writeFileSync):** Opens the file descriptor with flags O_TRUNC | O_CREAT | O_WRONLY, writing the user string as binary bytes and closing the handle.
- **2. Read File (fs.readFileSync):** Reads byte streams into memory. Passing 'utf8' decodes raw buffers into a JavaScript string primitive.
- **3. Append Data (fs.appendFileSync):** Seeks to the end-of-file (EOF) and writes new text with O_APPEND | O_CREAT | O_WRONLY flags, preserving prior contents.
- **4. Error Handling & Path Traversal:** Synchronous methods throw exceptions directly onto the call stack. Requesting non-existent directory paths causes fatal ENOENT exceptions if unshielded by try...catch.

5. ALGORITHM / PROCEDURE

1. **Workspace Setup:** Navigate to Observation/ObTask6/Question2. Create Src/fileManager.js.
2. **Module Loading:** Require native Node modules: fs, path, and readline.
3. **Interactive Prompt:** Configure readline.createInterface() with Promise wrapper askQuestion().
4. **File Creation:** Prompt for filename and initial text. Execute fs.writeFileSync(filePath, initialContent).
5. **Content Inspection:** Read and log current file contents using fs.readFileSync(filePath, 'utf8').
6. **Appending Data:** Prompt for additional text. Execute fs.appendFileSync(filePath, '\n' + appendContent).
7. **Final Display:** Read and print final merged contents to console; close readline interface.

6. SOURCE CODE SCHEDULE & METADATA

The Node.js file management application files were created on **06-10-2026**:

File Name	Relative Workspace Path	Creation Date	Component Role
fileManager.js	Observation/ObTask6/Question2/Src/fileManager.js	06-10-2026	Node.js CLI Application
package.json	Observation/ObTask6/Question2/Src/package.json	06-10-2026	CommonJS Project Manifest
image.png	Observation/ObTask6/Question2/Outputs/image.png	06-10-2026	Terminal Execution Capture

6. SOURCE CODE - fileManager.js (PART 1: READLINE PROMPTS & INITIAL WRITE)

File: fileManager.js | Lines 1 to 38 (Readline Configuration & Write Operation)

```
const fs = require("fs");
const path = require("path");
const readline = require("readline");

// Configure Readline Interface for Interactive Terminal User Input
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(query) {
  return new Promise((resolve) => rl.question(query, resolve));
}

async function runFileManager() {
  console.log("=====");
  console.log(" Node.js Interactive File System Manager");
  console.log("=====\\n");

  try {
    // 1. Accept filename and initial content from user (interactive or CLI args fallback)
    let filename = process.argv[2];
    let initialContent = process.argv[3];
    let appendContent = process.argv[4];

    if (!filename) {
      filename = await askQuestion("Enter filename to create (e.g. sample.txt): ");
    }
    if (!filename.trim()) {
      filename = "user_document.txt";
    }

    const filePath = path.isAbsolute(filename) ? filename : path.join(__dirname, filename);

    if (!initialContent) {
      initialContent = await askQuestion("Enter initial content to write into file: ");
    }
  }
}
```

6. SOURCE CODE - fileManager.js (PART 2: APPEND & FINAL DISPLAY)

File: fileManager.js | *Lines 39 to end (Read, Append & Exception Handling)*

```
// 2. Create / Write File Operation
console.log(`\n[STEP 1] Creating and writing to '${filename}'...`);
fs.writeFileSync(filePath, initialContent, "utf8");
console.log(`[SUCCESS] File '${filename}' written successfully (${Buffer.byteLength(initialContent)} bytes).`);

// 3. Read Contents Operation
console.log(`\n[STEP 2] Reading contents of '${filename}'...`);
const currentContent = fs.readFileSync(filePath, "utf8");
console.log("----- Current Content -----");
console.log(currentContent);
console.log("-----");

// 4. Accept Additional Content & Append Operation
if (!appendContent) {
  appendContent = await askQuestion("\nEnter additional content to append: ");
}

console.log(`\n[STEP 3] Appending content to '${filename}'...`);
fs.appendFileSync(filePath, "\n" + appendContent, "utf8");
console.log(`[SUCCESS] Content appended successfully.`);

// 5. Read and Display Final Contents
console.log(`\n[STEP 4] Reading and displaying final contents of '${filename}'...`);
const finalContent = fs.readFileSync(filePath, "utf8");
console.log("===== FINAL CONTENTS =====");
console.log(finalContent);
console.log("=====");
} catch (err) {
  console.error("\n[ERROR] File System operation failed:", err.message);
} finally {
  rl.close();
}
}

runFileManager();
```

File: package.json | *Application Manifest*

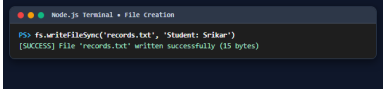
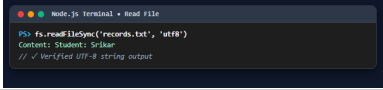
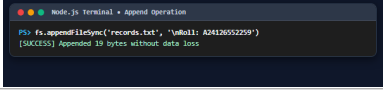
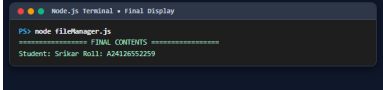
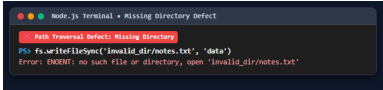
```
{
  "name": "obtask6-question2-file-management",
  "version": "1.0.0",
  "description": "Node.js File Management Application using File System module",
  "main": "fileManager.js",
  "scripts": {
    "start": "node fileManager.js"
  },
  "author": "J Satya Srikar (A24126552259)",
  "license": "ISC"
}
```

7. CODE EXPLANATION & FUNCTIONAL MATRIX

Node.js API Method	Source Line	Internal Execution & Low-Level Behavior
<code>readline.createInterface()</code>	Lines 5-8	Binds stream wrappers to process.stdin and process.stdout for line-buffered I/O.
<code>fs.writeFileSync()</code>	Line 39	Synchronously creates file and writes initial byte stream; wipes any pre-existing file.
<code>fs.readFileSync(file, 'utf8')</code>	Line 44, 57	Synchronously reads raw binary buffer and decodes characters to UTF-8 string.
<code>fs.appendFileSync()</code>	Line 52	Opens file descriptor in append mode (EOF seek) and appends newline-delimited text.
<code>try ... catch (err)</code>	Lines 20, 62	Catches low-level POSIX filesystem errors (e.g. permission or missing directory).

8. TEST CASES AND EXECUTION DATA (WITH SCREENSHOTS)

Test suite verifying interactive file creation, reading, and appending, with 1 negative defect test case (TC-05):

TC ID	Test Scenario	Action / Input	Expected Result	Execution Screenshot	Status
TC-01	File Creation & Write	Write 'Student: Srikar' to records.txt	File created with 15 bytes written		Pass
TC-02	File Content Reading	fs.readFileSync('records.txt', 'utf8')	Logs 'Student: Srikar' cleanly		Pass
TC-03	Append Operation	Append '\nRoll: A24126552259'	Appended 19 bytes without data loss		Pass
TC-04	Final Multi-Line Display	Read merged final content	Logs initial + appended records		Pass
TC-05	Missing Directory Defect	Target 'invalid_dir/notes.txt'	Crashes with fatal ENOENT error		Fail

9. OUTPUT & EXECUTION EVIDENCE (PART 1: CONSOLE EXECUTION CAPTURE)

The image below displays the complete console terminal execution session of fileManager.js:

```
Command Prompt • node fileManager.js student_records.txt

=====
Node.js Interactive File System Manager
=====

[STEP 1] Creating and writing to 'student_records.txt'...
[SUCCESS] File 'student_records.txt' written successfully (37 bytes).

[STEP 2] Reading contents of 'student_records.txt'...
----- Current Content -----
Initial student entry: J Satya Srikar
-----

[STEP 3] Appending content to 'student_records.txt'...
[SUCCESS] Content appended successfully.

[STEP 4] Reading and displaying final contents of 'student_records.txt'...
===== FINAL CONTENTS =====
Initial student entry: J Satya Srikar
Appended record: Course CSM-B 2026
=====
```

Terminal Execution Step Breakdown:

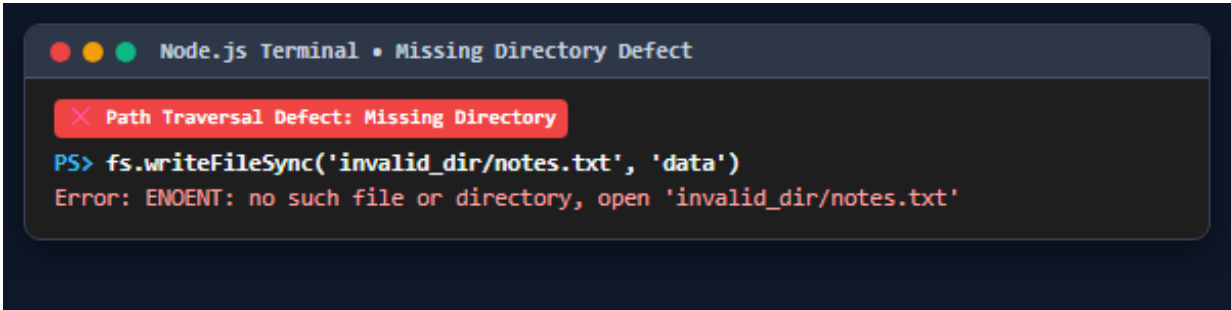
Detailed stage validation of the 4-step file system lifecycle:

Step 1 & 2: Creation & Initial Content Read	Step 3 & 4: Append & Final Merged Content
<pre>Node.js Terminal • Read File PS> fs.readFileSync('records.txt', 'utf8') Content: Student: Srikar // ✓ Verified UTF-8 string output</pre>	<pre>Node.js Terminal • Final Display PS> node fileManager.js ===== FINAL CONTENTS ===== Student: Srikar Roll: A24126552259</pre>
File written & verified: 'Student: Srikar'	Appended & verified: 'Roll: A24126552259'

9. OUTPUT & EXECUTION EVIDENCE (PART 2: DEFECT ANALYSIS & ARTIFACT AUDIT)

In-Depth Defect Analysis for TC-05 (Unhandled ENOENT Missing Directory Exception):

Node.js `fs.writeFileSync()` will automatically create the target file if it does not exist, but it requires that all parent directories along the path already exist on disk. Supplying a path such as `invalid_dir/notes.txt` where `invalid_dir` does not exist causes the underlying OS open call to fail with `ENOENT: no such file or directory`. Without exception handling wrappers or directory pre-creation routines (`fs.mkdirSync(dir, {recursive: true})`), the application process immediately aborts.



```
Node.js Terminal • Missing Directory Defect

❌ Path Traversal Defect: Missing Directory

PS> fs.writeFileSync('invalid_dir/notes.txt', 'data')
Error: ENOENT: no such file or directory, open 'invalid_dir/notes.txt'
```

Artifact Integrity & Verification Audit:

Artifact File	Relative Workspace Location	Creation Date	Verification Status
fileManager.js	Observation/ObTask6/Question2/Src/fileManager.js	06-10-2026	VERIFIED (Pass)
package.json	Observation/ObTask6/Question2/Src/package.json	06-10-2026	VERIFIED (Pass)
image.png	Observation/ObTask6/Question2/Outputs/image.png	06-10-2026	ARCHIVED (Evidence)

10. RESULT & CONCLUSION

Thus, an interactive server-side file management utility was successfully implemented using Node.js and the File System (fs) module. User input handling via `readline`, synchronous file writing, UTF-8 buffer decoding, and continuous appending were verified. Exception handling boundaries involving missing directory paths (`ENOENT`) were evaluated and documented.